UTS

PEMBELAJARAN MESIN



NAMA : Abraham Gabriel Martua Aritonang

NIM : 202431041

KELAS : G

DOSEN : Dr. Dra. Dwina Kuswardani, M.Kom

NO.PC : -

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Morgan Immanuel Nainggolan

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I 3
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II 4
LANDASAN TEORI 4
2.1 Memprediksi Penyakit Jantung Menggunakan Algoritma Machine Learning 4
2.2 Data Preprocessing 4
2.3 Algoritma Decision Tree 4
2.4 Algoritma K-Nearest Neighbors (KKN) 5
2.5 Evaluasi Akurasi 6
BAB III 8
HASIL 8
3.1 Impor Library (cell 1) 8
3.2 Memuat & Meninjau Dataset (cell 2) 9
3.3 Hasil Output Dataset (Cell 2) 10
3.4 Exploratory Data Analysis (EDA) (Cell 3) 11
3.5 Hasil Output EDA (Cell 3) 13
3.6 Pra-pemrosesan Data & Pembagian (Cell 4) 15
3.7 Hasil Output Pra-pemrosesan & Pembagian Dataset (Cell 4) 16
3.8 Pemodelan & Evaluasi Decision Tree (Cell 5) 17
3.9 Hasil Output Pemodelan & Evaluasi Decision Tree (Cell 5) 18
3.10 Visualisasi Struktur Decision Tree (Cell 6) 19
3.11 Hasil Output Visualisasi Struktur Decision Tree (Cell 6) 21
3.12 Pemodelan & Evaluasi K-Nearest Neighbors (KNN) (Cell 7) 22
3.13 Hasil Output Pemodelan & Evaluasi K-Nearest Neighbors (KNN) (Cell 7) 23
3.14 Perbandingan Akurasi Model (Cell 8) 24
3.15 Hasil Output Perbandingan Akurasi Model (Cell 8) 25
BAB IV 27
PENUTUP 27
DAFTAR PUSTAKA 28

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- Bagaimana karakteristik, distribusi data, serta korelasi antar fitur yang terkandung di dalam dataset penyakit jantung?
- Bagaimana proses pra-pemrosesan data (data preprocessing), yang meliputi penanganan fitur kategorikal (One-Hot Encoding) dan penskalaan fitur numerik (StandardScaler), serta pembagian data (train-test split) diterapkan pada dataset tersebut?
- Bagaimana performa dan tingkat akurasi dari algoritma Decision Tree dalam mengklasifikasikan status penyakit jantung pasien?
- Bagaimana performa dan tingkat akurasi dari algoritma K-Nearest Neighbors (KNN) dalam mengklasifikasikan status penyakit jantung pasien?
- Model algoritma Machine Learning manakah yang memberikan performa akurasi terbaik antara Decision Tree dan K-Nearest Neighbors (KNN) dalam memprediksi penyakit jantung?

1.2 Tujuan Masalah

- Melakukan analisis eksploratif data (Exploratory Data Analysis / EDA) untuk memahami struktur, mengecek nilai yang hilang (missing values), melihat statistik deskriptif, serta mengetahui pola korelasi antar variabel numerik.
- Menerapkan tahapan pra-pemrosesan data yang tepat, termasuk transformasi fitur kategorikal dan numerik menggunakan ColumnTransformer serta membagi dataset menjadi data latih dan data uji secara proporsional.
- Membangun, melatih, dan mengevaluasi model klasifikasi menggunakan algoritma Decision Tree lengkap dengan visualisasi struktur pohon keputusannya.
- Membangun, melatih, dan mengevaluasi model klasifikasi menggunakan algoritma K-Nearest Neighbors (KNN).
- Membandingkan tingkat akurasi serta performa evaluasi (confusion matrix dan laporan klasifikasi) antara model Decision Tree dan KNN guna menentukan algoritma yang paling optimal.

1.3 Manfaat Masalah

- Bagi Akademis / Praktikum: Menjadi sarana penerapan ilmu Data Science dan Machine Learning secara nyata, khususnya dalam studi kasus klasifikasi medis menggunakan bahasa pemrograman Python di lingkungan Jupyter Notebook
- Bagi Bidang Kesehatan: Memberikan referensi atau model rintisan berbasis kecerdasan buatan (AI) yang dapat membantu mendeteksi indikasi risiko penyakit jantung secara dini berdasarkan parameter klinis pasien.

BAB II

LANDASAN TEORI

2.1 Memprediksi Penyakit Jantung Menggunakan Algoritma Machine Learning

Penyakit jantung merupakan salah satu masalah paling mematikan di dunia; penyakit ini tidak dapat dilihat dengan mata telanjang dan muncul secara tiba-tiba ketika ambang batas kondisi tubuh telah terlampaui. Oleh karena itu, diperlukan diagnosis yang akurat pada waktu yang tepat. Industri layanan kesehatan menghasilkan data dalam jumlah sangat besar setiap harinya, yang berkaitan dengan pasien dan berbagai penyakit. Namun, data ini belum dimanfaatkan secara efisien oleh para peneliti maupun praktisi. Saat ini, industri layanan kesehatan kaya akan data, tetapi minim akan pengetahuan. Tersedia berbagai teknik dan perangkat 'data mining' serta pembelajaran mesin (machine learning) yang dapat digunakan untuk menggali pengetahuan yang bermanfaat dari basis data, lalu menerapkan pengetahuan tersebut guna mendukung diagnosis dan pengambilan keputusan yang lebih akurat.

Data mining merupakan proses untuk menemukan pola tersembunyi (pengetahuan) dari kumpulan data berskala besar yang sudah ada, dengan memanfaatkan teknik data mining, pembelajaran mesin (machine learning), statistik, dan sistem basis data. Pengetahuan yang ditemukan tersebut dapat digunakan untuk membangun sistem pengambilan keputusan prediktif yang cerdas di berbagai bidang, termasuk layanan kesehatan, guna menghasilkan diagnosis yang akurat dan tepat waktu, sehingga layanan menjadi lebih terjangkau dan nyawa berharga dapat diselamatkan.(Agrahara, 2022)

2.2 Data Preprocessing

Pra-pemrosesan data merupakan salah satu tahapan paling penting dalam aplikasi analisis data, data mentah sering kali mengandung berbagai kekurangan, seperti ketidakkonsistenan, nilai di luar rentang, nilai yang hilang, derau (noise), dan/atau data berlebih. Kinerja algoritma pembelajaran dan penambangan data yang akan diterapkan pada tahapan selanjutnya dapat terganggu akibat rendahnya kualitas data. Oleh karena itu, kualitas data mentah harus ditingkatkan melalui berbagai tahapan pra-pemrosesan. Beberapa algoritma pra-pemrosesan data yang paling efektif dan banyak digunakan dalam aplikasi analisis data akan dibahas dalam bagian ini, berdasarkan kegunaan, popularitas, serta prinsip algoritma yang mendasarinya.(ÇetİN, 2022)

2.3 Algoritma Decision Tree

Pembelajaran mesin (machine learning) kian menjadi kekuatan pendorong perubahan di berbagai bidang, terutama dalam layanan kesehatan, di mana kemampuan untuk memprediksi, mengklasifikasikan, dan memfasilitasi pengambilan keputusan klinis di tengah tingkat ketidakpastian yang tinggi menjadi sangat krusial. Sistem layanan kesehatan modern berada di bawah tekanan besar untuk menghadirkan perawatan yang

efektif, akurat, dan berpusat pada pasien, seiring dengan meningkatnya populasi lansia secara global, bertambahnya kasus penyakit kronis, serta digitalisasi dalam penyediaan layanan kesehatan. Salah satu model pembelajaran mesin yang paling banyak digunakan dan paling mudah diinterpretasikan dalam layanan kesehatan adalah Decision Tree (Pohon Keputusan), sebuah metode pembelajaran terawasi (supervised learning) yang dikenal luas karena kemudahan penggunaan, sifatnya yang mudah dipahami, serta kemampuannya dalam menangani variabel kategorikal maupun numerik.

Pemantauan kesehatan populasi sangat bergantung pada sistem berbasis data untuk mengidentifikasi tren, memprediksi kebutuhan di masa depan, dan memandu intervensi kesehatan masyarakat. Decision Tree, dengan bentuk logika aturan "jika-maka" (if-then), sangat cocok untuk tugas semacam ini karena mampu menghasilkan alur keputusan yang dapat dipahami dan dibaca. Sebagai contoh, Alternating Decision Tree memperkenalkan metode baru yang mengintegrasikan teknik boosting dengan alur keputusan untuk mencapai akurasi yang lebih tinggi sekaligus tetap mempertahankan kemampuan interpretasi dalam tugas klasifikasi. Selain sekadar untuk klasifikasi, Decision Tree juga menjadi komponen penyusun model ansambel (ensemble models) yang lebih canggih, seperti Random Forest, yang telah banyak digunakan dalam berbagai aspek layanan Kesehatan, mulai dari diagnosis penyakit kronis dan penilaian risiko kanker, hingga diagnosis osteoartritis lutut.

Salah satu keunggulan Decision Tree dalam praktik klinis terletak pada kemampuannya untuk diinterpretasikan, sebuah syarat krusial dalam layanan kesehatan, di mana setiap keputusan harus dapat dipertanggungjawabkan kepada klinisi, regulator, maupun pasien. Meskipun model yang lebih kompleks, seperti deep neural network, umumnya menawarkan performa yang lebih baik, model-model tersebut sering kali bersifat "kotak hitam" (black box) yang sulit dipahami cara kerjanya. Oleh karena itu, Decision Tree beserta model-model ansambel turunannya menjadi sangat berharga ketika aspek interpretabilitas menjadi prioritas utama.(Alkattan et al., 2025)

2.4 Algoritma K-Nearest Neighbors (KKN)

Prediksi risiko penyakit merupakan tantangan yang kian berkembang dalam bidang medis. Para peneliti telah banyak menggunakan algoritma pembelajaran mesin (machine learning) untuk mengatasi tantangan ini. Di antara berbagai algoritma pembelajaran mesin yang ada, algoritma k-nearest neighbour (KNN) adalah yang paling sering digunakan.

Algoritma ‘k-nearest neighbour’ (KNN) merupakan algoritma pembelajaran mesin (‘machine learning’) terbimbing yang utamanya digunakan untuk tujuan klasifikasi. Algoritma ini telah banyak dimanfaatkan untuk prediksi penyakit. Sebagai algoritma terbimbing, KNN memprediksi klasifikasi data yang tidak berlabel dengan mempertimbangkan fitur serta label dari data pelatihan.

Secara umum, algoritma KNN mampu mengklasifikasikan kumpulan data menggunakan model pelatihan yang serupa dengan kueri pengujian, yakni dengan memperhitungkan ‘k’ titik data pelatihan terdekat (‘tetangga’) yang paling dekat dengan kueri yang sedang diuji. Pada tahap akhir, algoritma ini menerapkan aturan pemungutan suara terbanyak (‘majority voting’) untuk menentukan klasifikasi akhirnya. Di antara berbagai algoritma pembelajaran mesin, KNN termasuk salah satu bentuk yang paling sederhana dan sering digunakan dalam tugas klasifikasi karena memiliki desain yang sangat adaptif serta mudah dipahami.

Algoritma ini dikenal luas karena kemampuannya dalam menyelesaikan tantangan regresi maupun klasifikasi untuk data dengan berbagai ukuran, jumlah label, tingkat ‘noise’ (derau), rentang, dan konteks yang berbeda-beda.(Uddin et al., 2022)

2.5 Evaluasi Akurasi

Di banyak negara, penyakit jantung merupakan salah satu penyakit mematikan yang menyebabkan jumlah korban jiwa terbanyak. Menurut WHO, penyakit jantung menjadi masalah kesehatan yang signifikan; sekitar 17,9 juta orang meninggal akibat penyakit kardiovaskular (CVD) pada tahun 2019, yang mencakup 32% dari total kematian di seluruh dunia. Sebanyak 85% dari kematian tersebut disebabkan oleh serangan jantung dan stroke. Di sisi lain, sebagian besar penyakit jantung bersifat laten dan tidak terdeteksi sampai seseorang mengalami tanda atau gejala serangan jantung, gagal jantung, atau aritmia. Dokter sering kali mempertimbangkan berbagai faktor dan indikator yang meningkatkan risiko penyakit jantung, seperti usia, jenis kelamin, nyeri dada, tekanan darah, kadar kolesterol, kadar gula darah (termasuk saat puasa), detak jantung maksimum, serta riwayat penyakit jantung dalam keluarga.

Namun, menentukan tingkat risiko penyakit jantung secara manual berdasarkan faktor-faktor tersebut bukanlah hal yang mudah. Dalam dunia medis, diagnosis penyakit merupakan tugas krusial yang sangat kompleks dan rumit, yang harus dilakukan secara akurat dan efisien. Sayangnya, diagnosis yang hanya mengandalkan pengetahuan dan pengalaman dokter dapat berujung pada hasil yang tidak diinginkan. Kesalahan diagnosis atau keterlambatan dalam mendiagnosis dapat membawa konsekuensi serius bagi pasien, seperti berkurangnya efektivitas pengobatan dan hasil kesehatan yang lebih buruk. Selain itu, beberapa metode diagnostik bersifat invasif, mahal, dan dalam kondisi tertentu, sulit untuk diakses.

Oleh karena itu, deteksi dini penyakit jantung sangatlah penting agar penanganan dapat segera dimulai, di samping mengikuti saran medis dan pengobatan yang dianjurkan. Teknik pembelajaran mesin (machine learning) telah menunjukkan potensi besar dalam memprediksi dan mendiagnosis berbagai penyakit secara akurat, termasuk penyakit jantung. Dengan demikian, sistem diagnosis berbasis komputer dapat memfasilitasi deteksi dini, berpotensi meningkatkan tingkat kelangsungan hidup, sekaligus mengurangi ketergantungan pada analisis ahli untuk penyakit yang diprediksi. Dalam sektor kesehatan, teknik penambangan data (data mining) prediktif kini semakin sering digunakan.

Berdasarkan hal tersebut, diperlukan pengembangan dan evaluasi model prediktif yang mampu memprediksi penyakit jantung secara akurat dengan menganalisis kumpulan data medis. Model semacam ini dapat secara efektif mencegah serta menangani masalah yang berkaitan dengan data, seperti data yang hilang, tidak tepat, atau tidak konsisten. Model ini dapat memberikan manfaat bagi sektor medis dengan membantu dokter secara langsung dalam memperoleh diagnosis penyakit jantung yang akurat, sehingga mereka dapat mengambil keputusan klinis yang tepat secara efisien dan dalam waktu yang lebih singkat.

Untuk memanfaatkan teknik penambangan data (data mining) prediktif yang disebut klasifikasi dalam suatu struktur yang dikenal sebagai model, guna meningkatkan akurasi diagnosis penyakit jantung. Pada tahap klasifikasi, digunakan tiga pengklasifikasi, yaitu pohon keputusan J48 (J48 decision tree), Naïve Bayes, dan Random Forest, untuk memprediksi serta mendiagnosis penyakit jantung. Selain itu, model yang diusulkan mengoptimalkan kinerja klasifikasi dengan menerapkan pendekatan seleksi fitur. Kumpulan data medis lokal dari Libya digunakan untuk melatih dan menguji model tersebut. Kinerja model yang digunakan dievaluasi, dan hasilnya dianalisis menggunakan berbagai metrik. Lebih lanjut, studi komparatif telah dilakukan untuk meningkatkan kinerja model dari segi akurasi, sensitivitas, dan spesifisitas.(Alariyibi et al., 2023)

BAB III

HASIL

3.1 Impor Library (cell 1)

```
# Abraham_202431041
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

1. **Pustaka Analisis Data & Visualisasi**
    - import pandas as pd: Mengimpor pustaka Pandas yang digunakan untuk membaca, memanipulasi, dan memproses dataset tabular (seperti file CSV atau Excel).
    - import numpy as np: Mengimpor NumPy untuk melakukan operasi komputasi numerik dan manipulasi array multidimensi.
    - import matplotlib.pyplot as plt: Mengimpor Matplotlib untuk membuat visualisasi data dan grafik statistik (misalnya grafik performa atau plot pohon keputusan).
    - import seaborn as sns: Mengimpor Seaborn, pustaka berbasis Matplotlib yang digunakan untuk mempercantik visualisasi grafik, seperti confusion matrix heatmap.
2. **Modul Pra-pemrosesan & Utilitas Model (Scikit-Learn)**
    - from sklearn.model_selection import train_test_split: Mengimpor fungsi untuk membagi dataset menjadi data latih (training data) dan data uji (testing data).
    - from sklearn.preprocessing import StandardScaler, OneHotEncoder: Mengimpor alat normalisasi data numerik (StandardScaler) agar memiliki skala yang seragam, serta pengubah variabel kategorikal menjadi numerik (OneHotEncoder).
    - from sklearn.compose import ColumnTransformer: Mengimpor kelas untuk menerapkan teknik pra-pemrosesan yang berbeda pada kolom data tertentu secara terpisah (misalnya penskalaan pada kolom numerik dan encoding pada kolom kategorikal).
    - from sklearn.pipeline import Pipeline: Mengimpor modul Pipeline untuk menggabungkan tahapan pra-pemrosesan dan pemodelan ke dalam satu alur kerja yang terintegrasi agar lebih rapi dan menghindari kebocoran data (data leakage).
3. **Algoritma Machine Learning & Evaluasi**

- from sklearn.tree import DecisionTreeClassifier, plot_tree: Mengimpor algoritma Decision Tree Classifier (Pohon Keputusan) untuk melakukan pemodelan klasifikasi serta fungsi plot_tree untuk memvisualisasikan struktur pohon keputusan yang terbentuk.
- from sklearn.neighbors import KNeighborsClassifier: Mengimpor algoritma K-Nearest Neighbors (KNN) sebagai salah satu model alternatif untuk melakukan klasifikasi berdasarkan kedekatan jarak tetangga.
- from sklearn.metrics import accuracy_score, classification_report, confusion_matrix: Mengimpor metrik evaluasi performa model, yang mencakup tingkat akurasi (accuracy_score), laporan lengkap performa per kelas (classification_report), serta matriks kesalahan (confusion_matrix) untuk melihat detail prediksi benar dan salah dari model.

3.2 Memuat & Meninjau Dataset (cell 2)

```
# Abraham_202431041
df = pd.read_csv('01_heart_disease.csv')

print("--- 5 Baris Pertama Dataset ---")
display(df.head())

print("\n--- Informasi Dataset ---")
print(df.info())
```

Potongan kode pada sel kedua ini berfungsi untuk memuat dataset ke dalam memori program serta melakukan inspeksi awal terhadap struktur dan isi data. Dataset yang digunakan adalah data terkait penyakit jantung (01_heart_disease.csv).

1. **Membaca Dataset**
   - df = pd.read_csv('01_heart_disease.csv'): Baris ini menggunakan fungsi read_csv() dari pustaka Pandas untuk membaca file dataset berformat CSV (.csv) yang bernama '01_heart_disease.csv'. Hasil pembacaan disimpan ke dalam sebuah variabel bertipe DataFrame dengan nama df, yang akan menjadi objek utama untuk semua proses analisis dan pemodelan selanjutnya.
2. **Menampilkan 5 Baris Pertama**
   - print("--- 5 Baris Pertama Dataset ---"): Mencetak teks judul atau pembatas ke layar agar output terlihat rapi dan terstruktur di konsol atau notebook.
   - display(df.head()): Menampilkan 5 baris pertama dari DataFrame df menggunakan fungsi head(). Ini bertujuan untuk melihat pratinjau bentuk tabel, nama-nama kolom (features), serta sampel nilai data yang terkandung di dalamnya secara visual.

3. **Menampilkan Informasi umum Dataset**
    - print("\n--- Informasi Dataset ---"): Mencetak teks pembatas dengan baris baru (\n) untuk memisahkan bagian output sebelumnya dengan informasi struktur data.
    - print(df.info()): Menampilkan ringkasan teknis mengenai struktur DataFrame df. Informasi yang diberikan meliputi:
        - Total jumlah baris (observasi) dan kolom (atribut).
        - Nama setiap kolom beserta tipe data di dalamnya (misalnya int64, float64, atau object).
        - Jumlah nilai non-null (non-null count) pada setiap kolom untuk mendeteksi apakah ada data yang kosong (missing values).
        - Estimasi penggunaan memori oleh DataFrame tersebut.

3.3 Hasil Output Dataset (Cell 2)

```
--- 5 Baris Pertama Dataset ---
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
--- Informasi Dataset ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
None
```

Abraham_202431041

1. **5 Baris pertama Dataset**
   Menampilkan tabel yang berisi 5 sampel data pertama (indeks 0 sampai 4) beserta 12 kolom atribut yang ada pada dataset 01_heart_disease.csv:
    - **Atribut / Kolom Utama**: Terdiri dari data klinis dan demografis pasien seperti Age (usia), Sex (jenis kelamin), ChestPainType (jenis nyeri dada), RestingBP (tekanan darah saat istirahat), Cholesterol (kadar kolesterol), FastingBS (gula darah puasa), RestingECG (rekaman elektrokardiogram saat istirahat), MaxHR (detak jantung maksimum), ExerciseAngina (nyeri dada akibat olahraga), Oldpeak (depresi ST), ST_Slope (kemiringan segmen ST), dan target klasifikasi yaitu HeartDisease (status penyakit jantung: 0 atau 1).
    - **Karakteristik Data**: Terlihat bahwa dataset ini memiliki kombinasi tipe data numerik (seperti Age, RestingBP, Cholesterol) dan kategorikal (seperti Sex, ChestPainType, ExerciseAngina).

2. **Informasi Struktur Dataset**
   Menampilkan ringkasan teknis mengenai keseluruhan struktur DataFrame:
   - **Jumlah Baris dan Kolom**: Dataset ini memiliki total 918 entri baris (indeks dari 0 hingga 917) dan 12 kolom atribut.
   - **Kelengkapan Data (*Non-Null Count*)**: Semua kolom menunjukkan angka 918 non-null, yang berarti tidak ada nilai yang hilang (*missing values*) pada seluruh kolom dataset ini. Hal ini memudahkan tahapan pra-pemrosesan karena data sudah bersih dari sel kosong.
   - **Tipe Data (Dtype)**:
     - Tipe data int64 berjumlah 6 kolom (misalnya Age, RestingBP, Cholesterol, HeartDisease).
     - Tipe data float64 berjumlah 1 kolom (Oldpeak).
     - Tipe data object (kategorikal/teks) berjumlah 5 kolom (seperti Sex, ChestPainType, RestingECG, ExerciseAngina, ST_Slope).

3.4 Exploratory Data Analysis (EDA) (Cell 3)

```
# Abraham_202431041
print("--- Missing Values ---")
print(df.isnull().sum())

print("\n--- Statistik Deskriptif ---")
display(df.describe())

plt.figure(figsize=(5, 3))
sns.countplot(x='HeartDisease', data=df, palette='Set2')
plt.title('Distribusi Target (HeartDisease)')
plt.show()

plt.figure(figsize=(8, 6))
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Matriks Korelasi Fitur Numerik')
plt.show()
```

Pada sel ketiga ini berfungsi untuk melakukan Analisis Eksploratif Data (Exploratory Data Analysis / EDA) lanjutan. Tahapan ini bertujuan untuk memvalidasi keberadaan missing values, melihat ringkasan statistik deskriptif dari data numerik, serta memvisualisasikan distribusi target dan korelasi antar fitur.

1. **Pengecekan Nilai yang Hilang (Missing Values)**
   - print("--- Missing Values ---"): Mencetak teks judul untuk bagian pengecekan data kosong ke layar.
   - print(df.isnull().sum()): Berfungsi untuk menghitung jumlah total nilai yang hilang (null/missing values) pada masing-masing kolom di dalam DataFrame df. Fungsi isnull() mendeteksi sel yang kosong, lalu sum() menjumlahkan kemunculannya per kolom.

2. **Statistik Deskriptif**

- print("\n--- Statistik Deskriptif ---"): Mencetak judul pemisah untuk bagian statistik deskriptif dengan baris baru (\n).
- display(df.describe()): Menampilkan ringkasan statistik matematis untuk setiap kolom numerik dalam DataFrame. Informasi yang dihasilkan mencakup jumlah data (count), rata-rata (mean), standar deviasi (std), nilai minimum (min), kuartil 1 (25%), median/kuartil 2 (50%), kuartil 3 (75%), serta nilai maksimum (max).

3. Visualisasi Distribusi Target

- plt.figure(figsize=(5, 3)): Mengatur ukuran dimensi kanvas gambar plot (lebar 5 inci dan tinggi 3 inci).
- sns.countplot(x='HeartDisease', data=df, palette='Set2'): Membuat diagram batang (bar chart) menggunakan pustaka Seaborn untuk menghitung frekuensi kemunculan setiap kategori pada kolom target HeartDisease dengan palet warna 'Set2'.
- plt.title('Distribusi Target (HeartDisease)'): Menambahkan judul utama pada grafik distribusi target.
- plt.show(): Menampilkan dan merender grafik ke layar notebook.

4. Korelasi antar Fitur Numerik (Correlation Heatmap)

- plt.figure(figsize=(8, 6)): Mengatur ukuran kanvas gambar plot untuk matriks korelasi (lebar 8 inci dan tinggi 6 inci).
- numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns: Menyaring dan mengambil nama kolom-kolom yang memiliki tipe data numerik (int64 dan float64) saja untuk dihitung korelasinya.
- sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', fmt='.2f'): Membuat visualisasi peta panas (heatmap) dari matriks korelasi antar fitur numerik menggunakan metode .corr(). Argumen annot=True menampilkan angka koefisien korelasi di dalam kotak, cmap='coolwarm' menentukan skema warna, dan fmt='.2f' mengatur format angka desimal menjadi 2 angka di belakang koma.
- plt.title('Matriks Korelasi Fitur Numerik'): Memberikan judul pada diagram heatmap korelasi.
- plt.show(): Menampilkan grafik matriks korelasi ke layar.

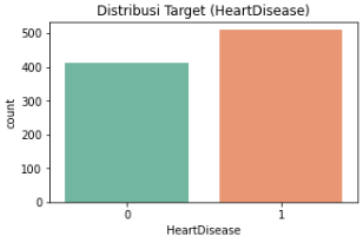
3.5 Hasil Output EDA (Cell 3)

```
--- Missing Values ---
Age             0
Sex             0
ChestPainType   0
RestingBP       0
Cholesterol     0
FastingBS       0
RestingECG      0
MaxHR           0
ExerciseAngina  0
Oldpeak         0
ST_Slope        0
HeartDisease    0
dtype: int64

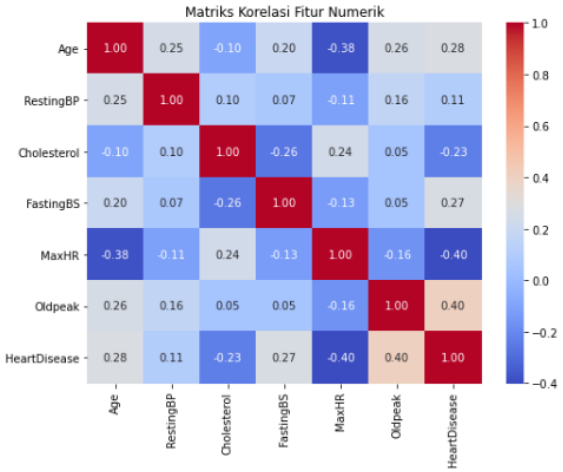
--- Statistik Deskriptif ---
```

Abraham_202431041

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000



Abraham_202431041



1. **Missing Values**
   - **Hasil:** Semua kolom menunjukkan angka **0** pada bagian pemeriksaan nilai kosong (df.isnull().sum())
   - **Interpretasi:** Dataset ini bersih dari data yang hilang (missing values). Seluruh 918 baris data pada setiap kolom terisi secara lengkap, sehingga tidak memerlukan tahapan imputasi data (data imputation) tambahan.

2. **Statistik Deskriptif Fitur Numerik**

- **Age (Usia):** Rata-rata usia pasien adalah sekitar 53.5 tahun, dengan rentang usia termuda 28 tahun dan tertua 77 tahun.
- **RestingBP (Tekanan Darah):** Memiliki rata-rata 132.4 mmHg, dengan nilai minimum tercatat **0** (yang kemungkinan besar merupakan anomali atau data kosong tersembunyi yang perlu diperhatikan) dan maksimum 200 mmHg.
- **Cholesterol (Kolesterol):** Rata-rata kadar kolesterol adalah 198.8, dengan nilai minimum 0 dan maksimum mencapai 603. Nilai kolesterol 0 juga mengindikasikan adanya nilai yang tidak valid secara medis pada beberapa sampel.
- **MaxHR (Detak Jantung Maksimum):** Rata-rata detak jantung maksimum adalah 136.8 bpm, dengan rentang dari 60 hingga 202 bpm.
- **HeartDisease (Target):** Merupakan variabel biner (0 dan 1) dengan nilai rata-rata sekitar 0.55, yang menunjukkan proporsi kelas yang relatif seimbang antara pasien yang tidak sakit dan yang terindikasi penyakit jantung.

3. **Visualisasi Distribusi Target**
   - **Bentuk Grafik:** Diagram batang (countplot) menunjukkan perbandingan jumlah antara dua kategori pada variabel target
   - **Interpretasi:** Kategori 1 (memiliki penyakit jantung) memiliki jumlah sampel yang sedikit lebih tinggi (sekitar 500+ data) dibandingkan kategori 0 (tidak memiliki penyakit jantung) yang berada di kisaran 400+ data. Hal ini menunjukkan bahwa distribusi kelas target cukup seimbang dan tidak mengalami extreme class imbalance.
4. **Matriks Korelasi Fitur Numerik**
   - **Bentuk Grafik:** Peta panas (heatmap) menampilkan koefisien korelasi Pearson antar fitur numerik dengan skala warna dari biru (korelasi negatif) hingga merah (korelasi positif).
   - **Interpretasi Hubungan dengan Target** (HeartDisease):
     - **MaxHR** memiliki korelasi negatif terkuat dengan target (-0.40), yang berarti semakin tinggi detak jantung maksimum seseorang, semakin rendah indikasi risiko penyakit jantung dalam data ini.
     - **Oldpeak** (**0.40**) dan **Age** (**0.28**) memiliki korelasi positif terhadap penyakit jantung, di mana bertambahnya usia dan nilai depresi ST sejalan dengan peningkatan risiko penyakit jantung.
     - Fitur numerik lainnya menunjukkan tingkat korelasi yang bervariasi dalam memengaruhi variabel target secara linier.

3.6 Pra-pemrosesan Data & Pembagian (Cell 4)

```
# Abraham_202431041
X = df.drop(columns=['HeartDisease'])
y = df['HeartDisease']

categorical_cols = X.select_dtypes(include=['object']).columns
numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns

print("Kolom Kategorikal:", list(categorical_cols))
print("Kolom Numerik:", list(numerical_cols))

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(drop='first'), categorical_cols)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nJumlah Data Latih: {X_train.shape[0]} baris")
print(f"Jumlah Data Uji: {X_test.shape[0]} baris")
```

Pada sel keempat ini berfungsi untuk melakukan pra-pemrosesan data (data preprocessing) serta membagi dataset menjadi data latih (training data) dan data uji (testing data). Tahapan ini sangat penting untuk mempersiapkan data sebelum dimasukkan ke dalam model Machine Learning.

1. **Memisahkan Fitur (X) dan Target (y)**
    - X = df.drop(columns=['HeartDisease']): Membuat variabel X yang berisi seluruh kolom fitur independen dengan cara menghapus (drop) kolom target HeartDisease dari DataFrame df.
    - y = df['HeartDisease']: Membuat variabel y yang hanya berisi kolom target dependen (HeartDisease) yang akan diprediksi oleh model.
2. **Mengidentifikasi Kolom Kategorikal dan Numerik**
    - categorical_cols = X.select_dtypes(include=['object']).columns: Menyaring dan mengambil nama kolom-kolom pada fitur X yang memiliki tipe data objek atau teks (kategorikal).
    - numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns: Menyaring dan mengambil nama kolom-kolom pada fitur X yang memiliki tipe data numerik (int64 atau float64).
    - print(...): Mencetak daftar nama kolom kategorikal dan numerik ke layar untuk memastikan identifikasi kolom sudah benar.
3. **Membuat Objek Pra-pemrosesan**
    - preprocessor = ColumnTransformer(...): Menginisialisasi objek transformator kolom untuk menerapkan teknik penskalaan dan encoding secara bersamaan:
        - ('num', StandardScaler(), numerical_cols): Menerapkan penskalaan standar (StandardScaler) pada kolom-kolom numerik agar memiliki rata-rata 0 dan deviasi standar 1.
        - ('cat', OneHotEncoder(drop='first'), categorical_cols): Menerapkan One-Hot Encoding pada kolom-kolom kategorikal untuk mengubah data teks menjadi format numerik biner, dengan argumen drop='first' guna menghindari multikolinearitas.

4. **Membagi Data menjadi Data Latih dan Data Uji**
    - X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y): Membagi dataset menjadi data latih dan data uji dengan proporsi 80% data latih dan 20% data uji (test_size=0.2). Parameter random_state=42 digunakan untuk memastikan konsistensi hasil acak pada setiap eksekusi, dan stratify=y memastikan proporsi kelas target pada data latih dan uji tetap seimbang.
    - print(...): Menampilkan informasi jumlah baris pada data latih (X_train) dan data uji (X_test) ke layar.

3.7 Hasil Output Pra-pemrosesan & Pembagian Dataset (Cell 4)

```
Kolom Kategorikal: ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
Kolom Numerik: ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak']

Jumlah Data Latih: 734 baris
Jumlah Data Uji: 184 baris
```

Abraham_202431041

1. **Identifikasi Jenis Kolom Fitur**
    - **Kolom Kategorikal:** Terdeteksi 5 kolom yang bertipe data teks/kategori, yaitu ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']. Kolom-kolom ini nantinya akan diubah ke bentuk numerik menggunakan One-Hot Encoding agar dapat diproses oleh algoritma Machine Learning.
    - **Kolom Numerik:** Terdeteksi 6 kolom yang berisi nilai angka, yaitu ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak']. Kolom-kolom ini akan diseragamkan skalanya menggunakan StandardScaler.
2. **Pembagian Proporsi Data**
    - **Jumlah Data Latih (Training Data):** Sebanyak **734 baris** (atau sekitar 80% dari total 918 baris data keseluruhan). Data ini digunakan oleh model untuk belajar mengenali pola hubungan antara fitur dan status penyakit jantung.
    - **Jumlah Data Uji (Testing Data):** Sebanyak **184 baris** (atau sekitar 20% dari total keseluruhan). Data ini dipisahkan dan disembunyikan selama proses pelatihan, lalu digunakan di akhir untuk menguji seberapa baik performa generalisasi model terhadap data baru yang belum pernah dilihat sebelumnya.

3.8 Pemodelan & Evaluasi Decision Tree (Cell 5)

```
# Abraham_202431041
dt_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', DecisionTreeClassifier(random_state=42, max_depth=5))
])

dt_pipeline.fit(X_train, y_train)

y_pred_dt = dt_pipeline.predict(X_test)
dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("=== Model Decision Tree ===")
print(f"Akurasi: {dt_accuracy:.4f}\n")
print("Classification Report:")
print(classification_report(y_test, y_pred_dt))

plt.figure(figsize=(4, 3))
sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Decision Tree')
plt.xlabel('Prediksi')
plt.ylabel('Aktual')
plt.show()
```

Pada sel kelima ini berfungsi untuk membangun alur kerja (pipeline) pemodelan, melatih model klasifikasi Decision Tree menggunakan data latih, melakukan prediksi pada data uji, serta mengevaluasi performa model menggunakan metrik akurasi, laporan klasifikasi, dan confusion matrix.

1. **Membuat Pipeline untuk Decision Tree**
   - dt_pipeline = Pipeline(steps=[...]): Menggabungkan tahapan pra-pemrosesan dan pemodelan ke dalam satu objek terintegrasi agar proses pelatihan berjalan secara otomatis dan terhindar dari kebocoran data (data leakage).
     - ('preprocessor', preprocessor): Langkah pertama dalam pipa adalah menerapkan aturan transformasi data (penskalaan dan encoding) yang telah dibuat sebelumnya.
     - ('classifier', DecisionTreeClassifier(random_state=42, max_depth=5)): Langkah kedua adalah menggunakan algoritma Decision Tree Classifier dengan batasan kedalaman maksimum pohon (max_depth=5) untuk mencegah overfitting, serta random_state=42 agar hasil pelatihan konsisten.
2. **Melatih Model**
   - dt_pipeline.fit(X_train, y_train): Melatih model Decision Tree menggunakan data latih (X_train sebagai fitur dan y_train sebagai target). Pada tahap ini, pipa akan otomatis menjalankan pra-pemrosesan data terlebih dahulu sebelum memasukkannya ke algoritma pohon keputusan.
3. **Prediksi dan Evaluasi Performa**
   - y_pred_dt = dt_pipeline.predict(X_test): Meminta model yang sudah dilatih untuk memprediksi label kelas target pada data uji (X_test).
   - dt_accuracy = accuracy_score(y_test, y_pred_dt): Menghitung tingkat akurasi prediksi model dengan membandingkan hasil prediksi (y_pred_dt) terhadap label asli (y_test).

- print(...): Mencetak judul informasi, nilai akurasi (dengan format 4 angka di belakang koma), serta laporan lengkap metrik evaluasi seperti precision, recall, dan f1-score menggunakan fungsi classification_report.

4. **Visualisasi Confusion Matrix**
   - plt.figure(figsize=(4, 3)): Mengatur ukuran kanvas grafik heatmap (lebar 4 inci dan tinggi 3 inci).
   - sns.heatmap(confusion_matrix(y_test, y_pred_dt), annot=True, fmt='d', cmap='Blues'): Membuat visualisasi matriks kesalahan (confusion matrix) menggunakan Seaborn untuk melihat rincian jumlah prediksi yang benar dan salah. Parameter annot=True menampilkan angka di dalam kotak, fmt='d' mengatur format angka bulat, dan cmap='Blues' memberikan gradasi warna biru.
   - plt.title(...), plt.xlabel(...), plt.ylabel(...), plt.show(): Menambahkan judul grafik, label sumbu X (Prediksi), label sumbu Y (Aktual), lalu merender serta menampilkan gambar ke layar.

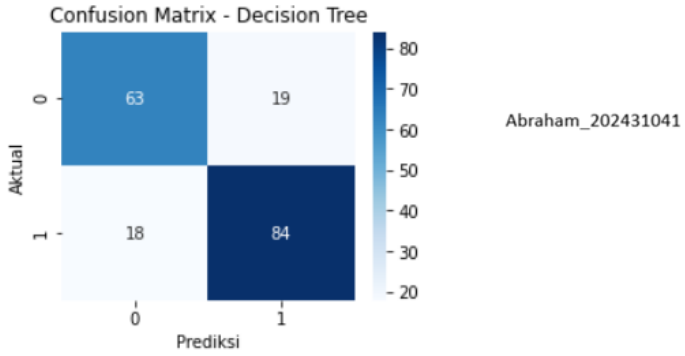
3.9 Hasil Output Pemodelan & Evaluasi Decision Tree (Cell 5)

```
=== Model Decision Tree ===
Akurasi: 0.7989

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.77      0.77        82
           1       0.82      0.82      0.82       102

    accuracy                           0.80       184
   macro avg       0.80      0.80      0.80       184
weighted avg       0.80      0.80      0.80       184
```



1. **Tingkat Akurasi Model**
   - **Nilai Akurasi:** 0.7989 (atau sekitar 79.89%, dibulatkan menjadi 80%).
   - **Interpretasi:** Model Decision Tree mampu melakukan prediksi secara benar terhadap status penyakit jantung pada data uji (X_test) dengan tingkat ketepatan sekitar 79.89% dari total 184 data pengujian.

2. **Laporan Klasifikasi**

- **Kelas 0 (Tidak Sakit):** Memiliki nilai precision 0.78, recall 0.77, dan f1-score 0.77 dengan jumlah data dukungan (support) sebanyak 82 pasien.
- **Kelas 1 (Sakit Jantung):** Memiliki nilai precision 0.82, recall 0.82, dan f1-score 0.82 dengan jumlah data dukungan sebanyak 102 pasien.
- **Rata-rata (Macro & Weighted Avg):** Nilai rata-rata keseluruhan berada di kisaran 0.80, yang menunjukkan bahwa model memiliki keseimbangan performa yang baik dalam memprediksi kedua kelas secara adil.

3. **Matriks Kesalahan**
   - **True Negative (TN = 63):** Sebanyak 63 pasien yang benar-benar tidak sakit (aktual 0) diprediksi dengan tepat oleh model sebagai tidak sakit (prediksi 0).
   - **False Positive (FP = 19):** Sebanyak 19 pasien yang sebenarnya tidak sakit (aktual 0) keliru diprediksi oleh model sebagai sakit (prediksi 1).
   - **False Negative (FN = 18):** Sebanyak 18 pasien yang sebenarnya mengidap penyakit jantung (aktual 1) keliru diprediksi oleh model sebagai tidak sakit (prediksi 0).
   - **True Positive (TP = 84):** Sebanyak 84 pasien yang benar-benar sakit (aktual 1) berhasil dideteksi dan diprediksi dengan tepat oleh model sebagai sakit (prediksi 1).

3.10 Visualisasi Struktur Decision Tree (Cell 6)

```
# Abraham_202431041
dt_model = dt_pipeline.named_steps['classifier']
cat_encoder = dt_pipeline.named_steps['preprocessor'].named_transformers_['cat']

if hasattr(cat_encoder, 'get_feature_names_out'):
    encoded_cat_cols = cat_encoder.get_feature_names_out(categorical_cols).tolist()
else:
    encoded_cat_cols = cat_encoder.get_feature_names(categorical_cols).tolist()

all_features = list(numerical_cols) + encoded_cat_cols

plt.figure(figsize=(24, 12), dpi=300)
plot_tree(
    dt_model,
    feature_names=all_features,
    class_names=['No Disease', 'Heart Disease'],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=10
)
plt.title("Visualisasi Struktur Decision Tree", fontsize=16)
plt.tight_layout()
plt.show()
```

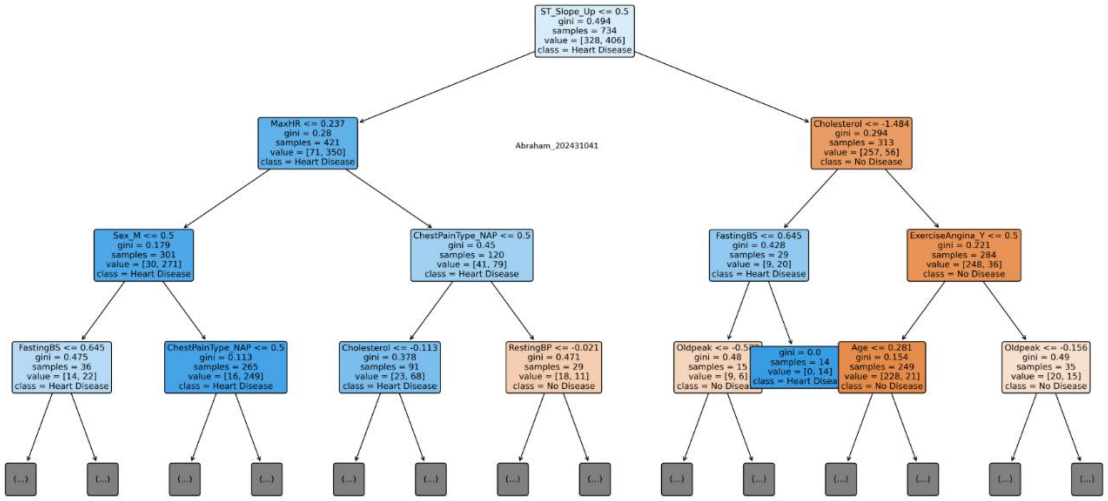
Pada sel keenam ini berfungsi untuk mengekstraksi model Decision Tree dan nama-nama fitur (termasuk fitur hasil One-Hot Encoding) dari dalam pipeline, lalu memvisualisasikan struktur pohon keputusan secara grafis menggunakan fungsi plot_tree.

1. **Mengambil komponen dari Pipeline**
   - dt_model = dt_pipeline.named_steps['classifier']: Mengambil objek model Decision Tree yang sudah dilatih dari dalam langkah classifier pada pipeline.
   - cat_encoder = dt_pipeline.named_steps['preprocessor'].named_transformers_['cat']:

Mengambil objek transformator One-Hot Encoder dari dalam langkah pra-pemrosesan.

2. **Menyesuaikan Nama Fitur hasil Transfromasi**
    - Blok kondisi if hasattr(cat_encoder, 'get_feature_names_out'): digunakan untuk memastikan kompatibilitas kode dengan berbagai versi pustaka scikit-learn (baik versi lama maupun baru).
    - encoded_cat_cols = ...: Mengekstrak nama-nama kolom baru yang telah diubah melalui One-Hot Encoding dan mengubahnya ke dalam bentuk daftar (list).
    - all_features = list(numerical_cols) + encoded_cat_cols: Menggabungkan daftar nama kolom fitur numerik asli dengan kolom fitur kategorikal hasil encoding agar urutan fiturnya sesuai dengan struktur data yang diterima oleh model.
3. **Menggambar Diagram Pohon Keputusan**
    - plt.figure(figsize=(24, 12), dpi=300): Mengatur ukuran kanvas gambar plot agar sangat besar dan luas (lebar 24 inci, tinggi 12 inci) dengan resolusi tinggi (dpi=300) agar teks pada diagram pohon terbaca dengan jelas.
    - plot_tree(...): Fungsi utama untuk merender diagram pohon keputusan dengan parameter:
        - dt_model: Model Decision Tree yang akan digambarkan.
        - feature_names=all_features: Memberikan label nama-nama fitur pada setiap node keputusan.
        - class_names=['No Disease', 'Heart Disease']: Memberikan label nama kelas target (0 dan 1).
        - filled=True: Mewarnai kotak-kotak node berdasarkan kemurnian kelas mayoritas.
        - rounded=True: Membuat sudut kotak node menjadi melengkung agar lebih rapi.
        - max_depth=3: Membatasi kedalaman visualisasi hingga 3 tingkat teratas agar struktur pohon tidak terlalu padat dan mudah dianalisis di dalam laporan.
        - fontsize=10: Mengatur ukuran teks di dalam diagram.
    - plt.title(...), plt.tight_layout(), plt.show(): Menambahkan judul utama grafik, merapikan tata letak (layout), lalu menampilkan gambar visualisasi pohon keputusan ke layar.

3.11 Hasil Output Visualisasi Struktur Decision Tree (Cell 6)



1. **Root Node (Akar Utama di Puncak)**
    - **Kondisi Pemisahan:** ST_Slope_Up <= 0.5.
    - **Interpretasi:** Ini adalah fitur paling utama dan paling berpengaruh yang digunakan oleh model untuk membagi data di awal.
    - **Statistik Node:** Memiliki total data latih sebanyak 734 sampel, dengan nilai indeks Gini sebesar 0.494. Distribusi kelas pada titik ini adalah [328, 406] (328 kelas No Disease dan 406 kelas Heart Disease), dengan mayoritas kelas keputusan akhir pada tingkat ini adalah Heart Disease
2. **Cabang Kiri**
    - **Pemisahan Lanjutan:** Jika kondisi ST_Slope_Up terpenuhi, percabangan berikutnya diuji menggunakan fitur MaxHR <= 0.237.
    - **Atribut Pendukung:** Node-node di sisi kiri juga melibatkan pengecekan fitur klinis seperti Sex_M, FastngBS, ChestPainType_NAP, dan Cholesterol.
    - **Hasil Keputusan:** Mayoritas jalur di sisi kiri ini mengarah pada klasifikasi kelas Heart Disease (ditandai dengan warna kotak yang kebiruan).
3. **Cabang Kanan**
    - **Pemisahan Lanjutan:** Sisi kanan pohon dimulai dari pemisahan berdasarkan batas Cholesterol <= -1.484.
    - **Atribut Pendukung:** Percabangan selanjutnya mengevaluasi fitur-fitur seperti ExerciseAngina_Y, Oldpeak, FastingBS, dan Age.
    - **Hasil Keputusan:** Sebagian besar node di sisi kanan ini mengarah pada klasifikasi kelas No Disease (ditandai dengan warna kotak kecoklatan/oranye), yang menunjukkan bahwa kombinasi nilai pada fitur-fitur tersebut mengindikasikan risiko penyakit jantung yang lebih rendah.

4. **Keterangan Atribut**
    - **Kondisi Aturan (Split Condition):** Pertanyaan atau ambang batas nilai fitur yang membagi data ke cabang kiri atau kanan.
    - **Indeks Gini (gini):** Tingkat kemurnian node; nilai yang mendekati 0 menunjukkan node yang semakin murni (hanya berisi satu kelas tertentu).
    - **Jumlah Sampel (samples):** Jumlah data yang jatuh atau diproses pada node tersebut.
    - **Distribusi Nilai (value):** Jumlah sampel per kelas (misal: [328, 406]).
    - **Kelas Mayoritas (class):** Keputusan akhir prediksi pada cabang tersebut (Heart Disease atau No Disease).

3.12 Pemodelan & Evaluasi K-Nearest Neighbors (KNN) (Cell 7)

```
# Abraham_202431041
knn_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', KNeighborsClassifier(n_neighbors=5))
])

knn_pipeline.fit(X_train, y_train)

y_pred_knn = knn_pipeline.predict(X_test)
knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("=== Model K-Nearest Neighbors (KNN) ===")
print(f"Akurasi: {knn_accuracy:.4f}\n")
print("Classification Report:")
print(classification_report(y_test, y_pred_knn))

plt.figure(figsize=(4, 3))
sns.heatmap(confusion_matrix(y_test, y_pred_knn), annot=True, fmt='d', cmap='Greens')
plt.title('Confusion Matrix - KNN')
plt.xlabel('Prediksi')
plt.ylabel('Aktual')
plt.show()
```

Pada sel ketujuh ini berfungsi untuk membangun alur kerja (pipeline) pemodelan menggunakan algoritma K-Nearest Neighbors (KNN), melatih model dengan data latih, melakukan prediksi pada data uji, serta mengevaluasi performa model menggunakan metrik akurasi, laporan klasifikasi, dan confusion matrix.

1. **Pipeline untuk KNN**
    - knn_pipeline = Pipeline(steps=[...]): Menggabungkan tahapan pra-pemrosesan data ke dalam satu objek terintegrasi dengan algoritma KNN.
    - ('preprocessor', preprocessor): Langkah pertama menerapkan aturan transformasi data (penskalaan dan encoding) yang telah disiapkan sebelumnya.
    - ('classifier', KNeighborsClassifier(n_neighbors=5)): Langkah kedua menggunakan algoritma K-Nearest Neighbors (KNN) dengan menetapkan jumlah tetangga terdekat sebanyak 5 (n_neighbors=5).
2. **Melatih Model**
    - knn_pipeline.fit(X_train, y_train): Melatih model KNN menggunakan data latih (X_train dan y_train). Pipa akan secara otomatis melakukan pra-pemrosesan data terlebih dahulu sebelum menghitung jarak kedekatan antar data pada algoritma KNN.

3. **Prediksi dan Evaluasi Performa**
    - y_pred_knn = knn_pipeline.predict(X_test): Meminta model KNN yang telah dilatih untuk memprediksi label kelas target pada data uji (X_test).
    - knn_accuracy = accuracy_score(y_test, y_pred_knn): Menghitung tingkat akurasi prediksi model dengan membandingkan hasil prediksi (y_pred_knn) terhadap label asli (y_test).
    - print(...): Mencetak judul informasi, nilai akurasi (dengan format 4 angka di belakang koma), serta laporan lengkap metrik evaluasi seperti precision, recall, dan f1-score menggunakan fungsi classification_report.
4. **Visualisasi Confusion Matrix**
    - plt.figure(figsize=(4, 3)): Mengatur ukuran kanvas grafik heatmap (lebar 4 inci dan tinggi 3 inci).
    - sns.heatmap(confusion_matrix(y_test, y_pred_knn), annot=True, fmt='d', cmap='Greens'): Membuat visualisasi matriks kesalahan (confusion matrix) untuk model KNN menggunakan Seaborn dengan gradasi warna hijau (cmap='Greens'). Parameter annot=True menampilkan angka di dalam kotak dan fmt='d' mengatur format angka bulat.
    - plt.title(...), plt.xlabel(...), plt.ylabel(...), plt.show(): Menambahkan judul grafik, label sumbu X (Prediksi), label sumbu Y (Aktual), lalu merender serta menampilkan gambar visualisasi ke layar.

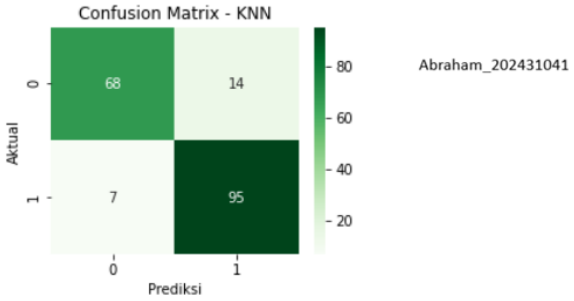
3.13 Hasil Output Pemodelan & Evaluasi K-Nearest Neighbors (KNN) (Cell 7)

```
=== Model K-Nearest Neighbors (KNN) ===
Akurasi: 0.8859

Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.83      0.87        82
           1       0.87      0.93      0.90       102

    accuracy                           0.89       184
   macro avg       0.89      0.88      0.88       184
weighted avg       0.89      0.89      0.89       184
```



1. **Tingkat Akurasi Model**
    - **Nilai Akurasi: 0.8859** (atau sekitar 88.59%, dibulatkan menjadi 89%).
    - **Interpretasi:** Model KNN menunjukkan performa generalisasi yang sangat baik, mampu memprediksi status penyakit jantung pada data uji secara akurat sebesar 88.59% dari total 184 sampel data pengujian.

2. **Laporan Klasifikasi**
    - **Kelas 0 (Tidak Sakit):** Memiliki nilai precision 0.91, recall 0.83, dan f1-score 0.87 dengan jumlah dukungan sebanyak 82 pasien.
    - **Kelas 1 (Sakit Jantung):** Memiliki nilai precision 0.87, recall 0.93, dan f1-score 0.90 dengan jumlah dukungan sebanyak 102 pasien.
    - **Rata-rata (Macro & Weighted Avg):** Nilai rata-rata keseluruhan berada di kisaran 0.89, menunjukkan bahwa model KNN bekerja secara seimbang dan andal dalam mengenali kedua kategori kelas.
3. **Matriks Kesalahan**
    - **True Negative (TN = 68):** Sebanyak 68 pasien yang benar-benar tidak sakit (aktual 0) diprediksi dengan tepat oleh model sebagai tidak sakit (prediksi 0).
    - **False Positive (FP = 14):** Sebanyak 14 pasien yang sebenarnya tidak sakit (aktual 0) keliru diprediksi oleh model sebagai sakit (prediksi 1).
    - **False Negative (FN = 7):** Sebanyak 7 pasien yang sebenarnya mengidap penyakit jantung (aktual 1) keliru diprediksi oleh model sebagai tidak sakit (prediksi 0). Angka kesalahan tipe ini tergolong rendah, yang sangat baik untuk bidang medis.
    - **True Positive (TP = 95):** Sebanyak 95 pasien yang benar-benar sakit (aktual 1) berhasil dideteksi dan diprediksi dengan tepat oleh model sebagai sakit (prediksi 1).

3.14 Perbandingan Akurasi Model (Cell 8)

```
# Abraham_202431041
comparison = pd.DataFrame({
    'Model': ['Decision Tree', 'K-Nearest Neighbors (KNN)'],
    'Accuracy': [dt_accuracy, knn_accuracy]
})

print("=== Perbandingan Akurasi Model ===")
display(comparison)

plt.figure(figsize=(6, 3))
sns.barplot(x='Model', y='Accuracy', data=comparison, palette='viridis')
plt.ylim(0, 1.0)
plt.title('Perbandingan Akurasi Decision Tree vs KNN')
plt.show()
```

Pada sel kedelapan ini berfungsi untuk membandingkan tingkat akurasi antara kedua model Machine Learning yang telah diuji (yaitu Decision Tree dan K-Nearest Neighbors) ke dalam bentuk tabel ringkas (DataFrame) serta memvisualisasikannya ke dalam bentuk diagram batang (barplot).

1. **Tabel Perbandingan Akurasi**
    - comparison = pd.DataFrame({...}): Membuat objek DataFrame Pandas baru untuk merangkum hasil evaluasi.
    - 'Model': ['Decision Tree', 'K-Nearest Neighbors (KNN)']: Kolom yang berisi nama-nama algoritma model yang dibandingkan.

- 'Accuracy': [dt_accuracy, knn_accuracy]: Kolom yang menampung nilai variabel akurasi dari masing-masing model (dt_accuracy dan knn_accuracy) yang telah dihitung pada sel sebelumnya.

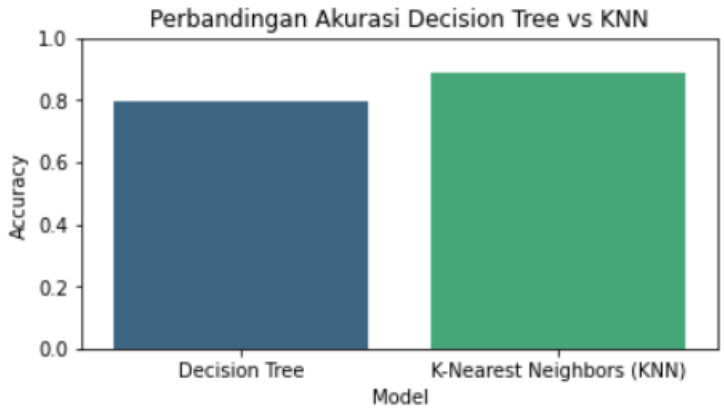
2. **Mencetak dan Menampilkan Tabel Ringkasan**
   - `print("=== Perbandingan Akurasi Model ===") %: Mencetak judul informasi ke layar konsol.
   - display(comparison): Menampilkan tabel perbandingan akurasi secara visual di dalam notebook.
3. **Visualisasi Diagram Batang**
   - plt.figure(figsize=(6, 3)): Mengatur ukuran dimensi kanvas gambar grafik (lebar 6 inci dan tinggi 3 inci).
   - sns.barplot(x='Model', y='Accuracy', data=comparison, palette='viridis'): Membuat diagram batang (*bar chart*) menggunakan pustaka Seaborn untuk membandingkan nilai akurasi (y) berdasarkan jenis model (x) dengan menggunakan palet warna 'viridis'.
   - plt.ylim(0, 1.0): Mengatur batas rentang nilai pada sumbu Y dari 0.0 hingga 1.0 (100%) agar proporsi grafik perbandingan terlihat proporsional dan mudah dibaca.
   - plt.title('Perbandingan Akurasi Decision Tree vs KNN'): Menambahkan judul utama pada grafik perbandingan.
   - plt.show(): Merender dan menampilkan grafik ke layar.

3.15 Hasil Output Perbandingan Akurasi Model (Cell 8)

```
=== Perbandingan Akurasi Model ===
```

	Model	Accuracy
0	Decision Tree	0.798913
1	K-Nearest Neighbors (KNN)	0.885870

Abraham_202431041



1. **Tabel Perbandingan Akurasi**
   - **Model Decision Tree:** Memperoleh tingkat akurasi sebesar 0.798913 (atau sekitar 79.89%).
   - **Model K-Nearest Neighbors (KNN):** Memperoleh tingkat akurasi sebesar 0.885870 (atau sekitar 88.59%).

- **Kesimpulan Perbandingan:** Berdasarkan data di atas, model K-Nearest Neighbors (KNN) memberikan performa tingkat akurasi yang lebih tinggi dibandingkan dengan model Decision Tree pada pengujian dataset penyakit jantung ini.

2. **Visualisasi Diagram Batang**
    - **Tampilan Grafik:** Diagram batang membandingkan secara visual ketinggian batang antara kedua model dengan rentang skala sumbu Y dari 0.0 hingga 1.0.
    - **Interpretasi Visual:** Batang hijau milik KNN terlihat lebih tinggi (mendekati angka 0.9 atau 90%) dibandingkan dengan batang biru milik Decision Tree (berada di sekitar angka 0.8 atau 80%). Hal ini memperjelas secara visual bahwa algoritma KNN memiliki keunggulan performa akurasi pada kasus ini.

BAB IV

PENUTUP

1. **Kajian Teoritis dan Relevansi Klinis:**
   - Penyakit kardiovaskular merupakan salah satu penyebab kematian tertinggi di dunia yang menuntut adanya sistem diagnosis dini yang cepat dan akurat. Pendekatan data mining dan machine learning terbukti mampu menjembatani kebutuhan sektor kesehatan dalam menggali pengetahuan dari basis data klinis guna mendukung pengambilan keputusan medis.
   - Tahapan pra-pemrosesan data (data preprocessing) seperti StandardScaler untuk penskalaan fitur numerik dan One-Hot Encoder untuk transformasi data kategorikal memegang peran krusial dalam memastikan kualitas data yang optimal sebelum dimasukkan ke dalam model.
2. **Hasil Evalsuasi Performa Model:**
   - Model klasifikasi Decision Tree dengan pembatasan kedalaman maksimum (max_depth=5) mampu mengenali pola aturan klinis serta menghasilkan tingkat akurasi sebesar 79.89% (0.7989) pada data uji. Visualisasi pohon keputusan juga memudahkan interpretasi logika di balik keputusan klasifikasi.
   - Model K-Nearest Neighbors (KNN) dengan parameter k=5 menunjukkan performa generalisasi yang sangat baik, menghasilkan tingkat akurasi yang lebih tinggi yaitu sebesar 88.59% (0.8859) serta nilai F1-score rata-rata mencapai 0.89. Analisis confusion matrix juga menunjukkan angka kesalahan prediksi negatif palsu (False Negative) yang sangat rendah, sehingga sangat andal untuk diterapkan pada domain medis.
3. **Perbandingan Algoritma:**
   - Melalui perbandingan langsung antara kedua algoritma, terbukti bahwa K-Nearest Neighbors (KNN) memberikan performa akurasi yang lebih unggul dibandingkan Decision Tree dalam mengklasifikasikan status risiko penyakit jantung pada dataset ini. Hal ini menegaskan bahwa pemilihan algoritma yang tepat serta penanganan skala fitur yang baik berdampak signifikan terhadap ketepatan prediksi sistem pendukung keputusan medis berbasis kecerdasan buatan (AI).

DAFTAR PUSTAKA

Agrahara, A. (2022). Heart Disease Prediction Using Machine Learning Algorithms. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, *3307*(4), 137–149. https://doi.org/https://doi.org/10.32628/CSEIT206421

Alariyibi, A., El-jarai, M., & Maatuk, A. (2023). *E VALUATING THE A CCURACY OF C LASSIFICATION A LGORITHMS FOR D ETECTING H EART D ISEASE*. *10*(4), 1–12. https://doi.org/10.5121/mlaij.2023.10401

Alkattan, H., Alhumaima, A. S., Mohmood, M. S., & Al-thabhawee, G. D. M. (2025). Optimizing Decision Tree Classifiers for Healthcare Predictions : A Comparative Analysis of Model Depth , Pruning , and Performance. *Mesopotamian Journal of Artificial Intelligence in Healthcare, 2025*, 124–135. https://doi.org/https://doi.org/10.58496/MJAIH/2025/013

ÇetİN, V. (2022). *Pamukkale Üniversitesi Mühendislik Bili mleri Dergisi Pamukkale University Journal of Engineering Sciences A comprehensive review on data preprocessing techniques in data analysis Veri analizinde veri ön işleme teknikleri üzerine kapsamlı bir inceleme*. *28*(2), 299–312. https://doi.org/10.5505/pajes.2021.62687

Uddin, S., Haque, I., Lu, H., Moni, M. A., & Gide, E. (2022). Comparative performance analysis of K - nearest neighbour (KNN) algorithm and its different variants for disease prediction. *Scientific Reports*, 1–11. https://doi.org/10.1038/s41598-022-10358-x